SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

ISO/IEC 15939 Measurement Process Simulator

---

1. Introduction

1.1 Purpose

This document defines the software requirements for a desktop application that simulates the ISO/IEC 15939 software measurement process. The application enables users to define quality metrics, collect measurement data, and analyze results through a structured 5-step workflow.

1.2 Scope

The system is a Java Swing-based desktop application. It allows users to:

- Enter profile information
- Select quality type, mode, and scenario
- View predefined quality dimensions and metrics
- Collect and process measurement data
- Analyze results using weighted scores and visual indicators

The system is intended for educational purposes in Software Engineering courses.

1.3 Definitions

- **Metric**: A measurable attribute of a system
- **Dimension**: A group of related metrics
- **Scenario**: A predefined dataset of dimensions and metrics
- **Score**: A normalized value between 1–5 calculated from raw data

---

2. Overall Description

2.1 Product Perspective

The system is a standalone desktop application. It follows a wizard-based structure where each step is displayed in sequence.

2.2 Product Functions

The system provides the following functions:

- User profile management
- Quality definition selection
- Scenario-based metric planning

- Data collection and score calculation
- Analysis and visualization of results
- Displaying a step indicator at the top of the screen to track process progress.

2.3 User Characteristics

- Basic computer literacy
- No technical expertise required

2.4 Constraints

- Must be implemented using Java SE and Swing
- No external libraries allowed
- Data is hard-coded (no database required)

---

3. System Features

3.1 Step 1 – Profile

Description

Collects user information.

Functional Requirements

- User must enter:
    - Username
    - School
    - Session Name
- System shall validate inputs:
    - All fields must be filled
    - If not, display a warning message

3.2 Step 2 – Define Quality

Description

User selects quality type, mode, and scenario.

Functional Requirements

- System shall allow selecting exactly one:
    - Quality Type (Product / Process)
- System shall allow selecting exactly one:

    - Mode (Custom / Health / Education)
- System shall dynamically display scenarios based on selected mode
- Each mode shall contain at least two scenarios

3.3 Step 3 – Plan Measurement

Description

Displays predefined dimensions and metrics.

Functional Requirements

- System shall display:
    - Dimension name and coefficient
    - Metric name
    - Metric coefficient
    - Direction (Higher/Lower)
    - Range
    - Unit
- Data shall be read-only

3.4 Step 4 – Collect Data

Description

Displays raw values and calculates scores.

Functional Requirements

- System shall display raw metric values
- System shall calculate scores as follows:

For “Higher is better”:
score = 1 + (value − min) / (max − min) × 4

For “Lower is better”:
score = 5 − (value − min) / (max − min) × 4

- Scores shall be:
    - Between 1.0 and 5.0
    - Rounded to nearest 0.5

3.5 Step 5 – Analyse

Description

Displays analysis results.

Functional Requirements

- System shall calculate dimension score:
  dimensionScore = Σ(metricScore × coefficient) / Σ(coefficient)
- System shall display results using progress bars
- System shall identify the lowest scoring dimension
- System shall display:
  - Dimension name
  - Score
  - Gap value (5 − score)
  - Quality level:
    - Excellent
    - Good
    - Needs Improvement
    - Poor

---

4. Non-Functional Requirements

4.1 Usability

- Interface must be simple and user-friendly
- Clear error messages must be displayed

4.2 Performance

- Application should respond instantly to user input

4.3 Reliability

- Calculations must be accurate

4.4 Maintainability

- Code must follow OOP principles

---

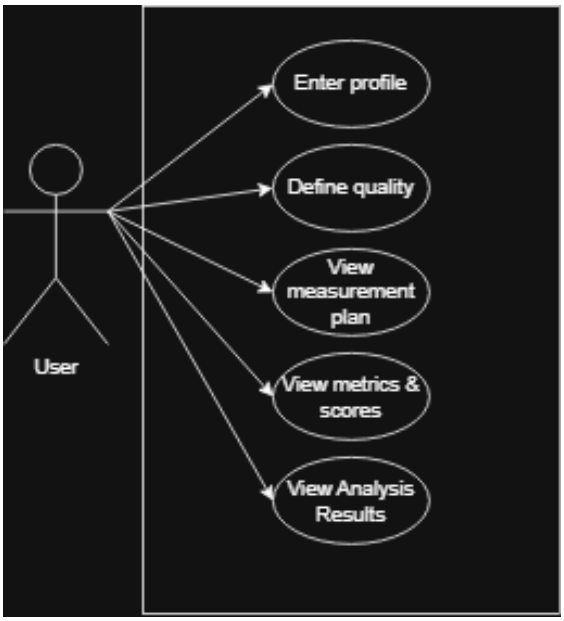
5. System Design

5.1 Architecture

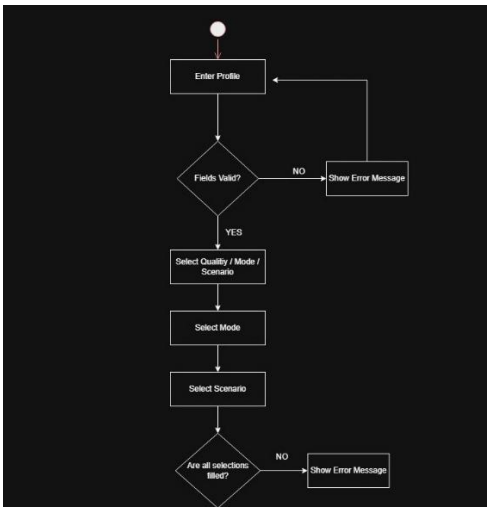
The system follows MVC (Model-View-Controller) architecture.

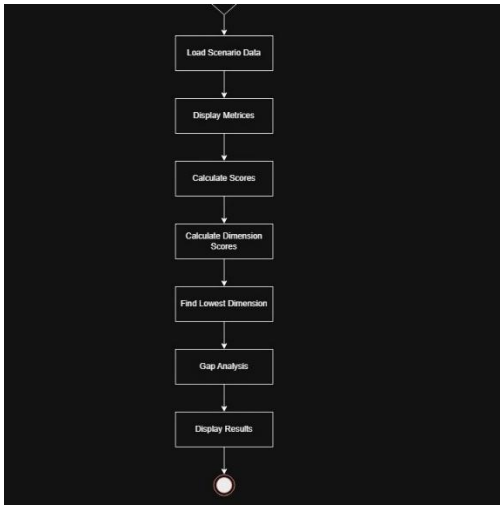
- Model: Metric, Dimension, Scenario classes
- View: Swing GUI components
- Controller: Handles logic and interactions

Use Case Diagram:



Activity Diagram:



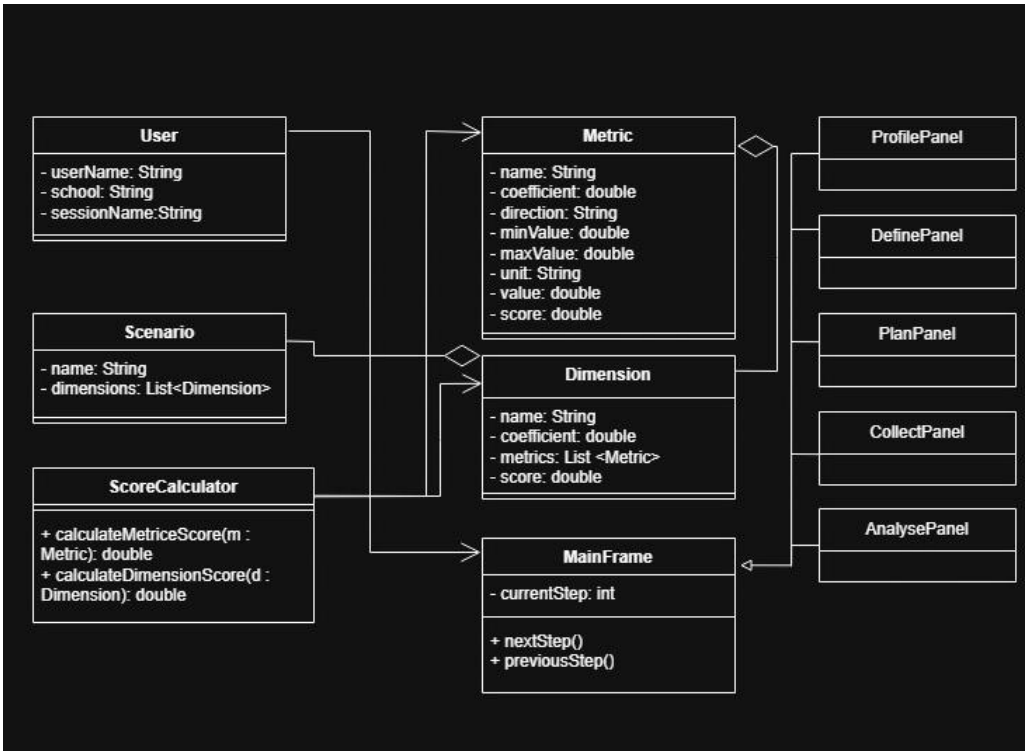


5.2 Class Design (Conceptual)

- User
- Metric
- Dimension

- Scenario
- ScoreCalculator
- MainFrame
- StepPanels (ProfilePanel, DefinePanel, etc.)

Class Diagram:



---

6. Assumptions and Dependencies

- All data is predefined
- No internet connection required
- No external systems integrated

---

7. Appendix

Example dataset includes:

- Usability
- Performance Efficiency
- Accessibility
- Reliability
- Functional Suitability

Each dimension contains multiple metrics with coefficients and ranges.